

Integrating logical constraints in one-class graph neural networks for anomaly detection

Anastasiia Romanenko

Technische Universität Dortmund
Fakultät Statistik

6. march 2026

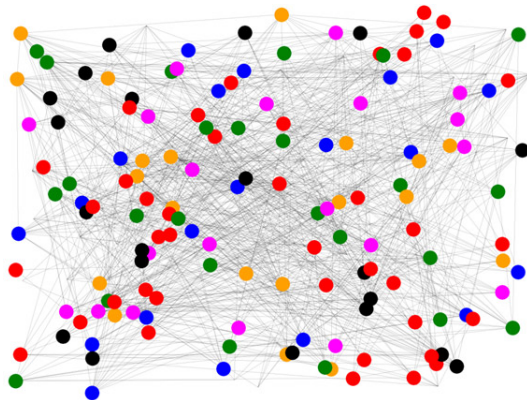
- 1 Problem statement
- 2 Existing graph anomaly detection methods
- 3 Existing Methods with Logical Constraints
- 4 My approach
 - data
 - Implementing logical rules into OCGIN

Section 1

Problem statement

Importance of Anomaly Detection on Graph Data

An anomaly is an unusual or unexpected pattern in data. Detecting anomalies is important for identifying unknown threats and irregular behavior in complex systems.



Motivation of constrains driven graph AD

In many real-world application we have prior knowledge about data, which can be formulated as logical rules

- If `person_height > 200 cm` $\rightarrow$ anomaly
- If a financial transaction graph contains a forbidden cycle pattern $\rightarrow$ anomaly
- If a molecular graph violates valence constraints $\rightarrow$ anomaly

Classical One-Class Graph Neural Networks:

- learn only from data
- do not explicitly learn logical rules
- may learn representations that violate known knowledge

Section 2

Existing graph anomaly detection methods

Anomalies on graph data

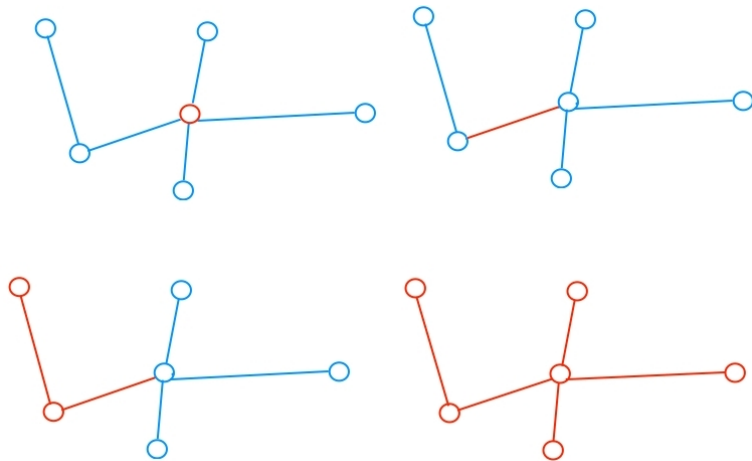


Figure 1: Kinds of anomalies on graph data

Previous One-Class Graph Methods

Common assumptions:

- Access to a large set of normal graph data only
- Learn a graph embedding $f_{\theta}(G) \in \mathbb{R}^d$
- Model “normality” as a compact region in the embedding space

Typical objective (hypersphere learning):

$$\min_{\theta} \sum_i \|f_{\theta}(G_i) - c\|^2$$

where c denotes the center of normal data.

OCGNN – Structure

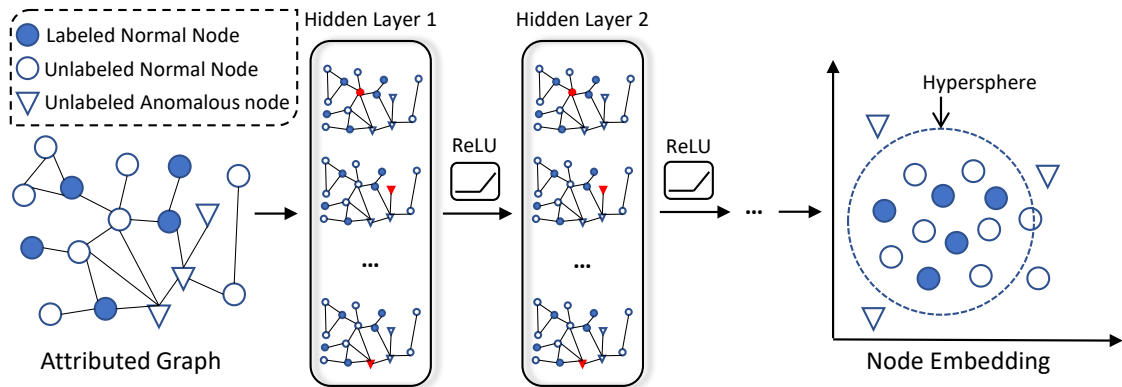


Figure 2: Structure of OCGIN¹

¹Wang et al. 2020.

OCGNN – Forward Pass

For a graph $G = (V, E)$ with node features $X \in \mathbb{R}^{|V| \times d_{in}}$:

$$h_v^{(k+1)} = \text{MLP}^{(k)} \left((1 + \epsilon) \cdot h_v^{(k)} + \sum_{u \in \mathcal{N}(v)} h_u^{(k)} \right)$$

where ϵ is a learnable parameter, and the MLP consists of Linear $\rightarrow$ ReLU $\rightarrow$ Linear layers.

$$z_i = \frac{1}{|V_i|} \sum_{v \in V_i} h_v^{(L)}$$

Global mean pooling aggregates the node embeddings into a graph-level embedding $z_i \in \mathbb{R}^{d_{hidden}}$.

$$L_{\text{OCGIN}} = \frac{1}{n} \sum_{i=1}^n \|z_i - c\|^2$$

where c is the center of normality (initialized as the mean of the normal data embeddings).

Section 3

Existing Methods with Logical Constraints

1) Constraints as Semantic Loss²

- Logical rules are manually encoded as differentiable loss terms
- The neural network is trained to minimize both the data loss and the constraint loss

2) Fuzzy Logic in Loss³

- Logical rules are relaxed by applying continuous functions

3) Constraint-aware Architectures⁴

- Logical rules compiled into network structure or output layer
- Guarantees or encourages valid structured outputs

²Xu et al. 2018.

³Badreddine et al. 2022.

⁴Yang, Lee, and Park 2023.

Common Assumptions

- Domain knowledge is available in the form of logical rules
- Constraints can be formulated as differentiable terms or probabilistically weighted

Empirical Benefits^{5, 6}

- Improved performance
- Models are able to learn from less data
- Increased interpretability through semantic rule satisfaction

⁵Giunchiglia, Mihaela Catalina Stoian, and Lukasiewicz 2022.

⁶Mihaela Cătălina Stoian et al. 2024.

Section 4

My approach

Why Are Graph AD Datasets Uncommon?

Problem with Collecting Graph AD Datasets:

- Anomalies are rare, diverse, and unknown by definition (typically $< 5\%$ of the data)
- They are difficult to collect and label

Common Solution: Pseudo-Labels

- Use labeled graph datasets where one class is treated as normal
- Treat the majority class as “normal” and the minority class as “anomaly”

Problem: Finding Logical Constraints

- Domain knowledge is required to define rules for each dataset
- Domain experts are not always available

Solution: Generate Constraints from Decision Trees

- 1 Add graph-related features to the data (e.g., node degree)
- 2 Train a decision tree on the (pseudo-)labeled data
- 3 Extract logical rules from the decision tree paths

Example: Constraint JSON

```
{
  "constrains": [
    {
      "conditions": [
        {"feature_index": 1263, "op": "<=", "threshold": 0.0},
        {"feature_index": 205, "op": "<=", "threshold": 0.0}
      ],
      "predicted_class": 1,
      "meta": {"depth": 2, "y_label_number": "98 v.s. 243"}
    },
    ...
  ],
  "additional_attributes": {
    "0:1433": "node_features",
    "1433": "node_degree",
    "1434": "clustering_coefficient"
  }
}
```

Hard Rules $\rightarrow$ differentiable fuzzy logic

- 1 **Feature Extraction:** Combine original features with derived features (e.g., node degree)
- 2 **Rule Evaluation:** For each anomaly rule:
 - Evaluate all conditions using soft logic
 - Combine them with soft AND
- 3 **Combine Rules:** Apply soft OR over all rules

Output: A scalar violation score per node (higher values indicate stronger constraint violation for a chosen class of constraints)

Comparison operators:

- $x \leq t \rightarrow \sigma(\lambda \cdot (t - x))$
- $x > t \rightarrow \sigma(\lambda \cdot (x - t))$

Logic gates:

- $A \wedge B \rightarrow \prod_i A_i$
- $A \vee B \rightarrow 1 - \prod_i (1 - A_i)$

Loss Functions in Node-Level OCGIN

Two variants have been implemented:

Variant 1 – Additive:

$$L_{\text{total}} = \frac{1}{n} \sum_{i=1}^n \|z_i - c\|^2 + \frac{1}{n} \sum_{i=1}^n L_{\text{cons}}(i)$$

Variant 2 – Weighting:

$$L_{\text{total}} = \frac{1}{n} \sum_{i=1}^n (\|z_i - c\|^2 * (1 + L_{\text{cons}}(i)))$$

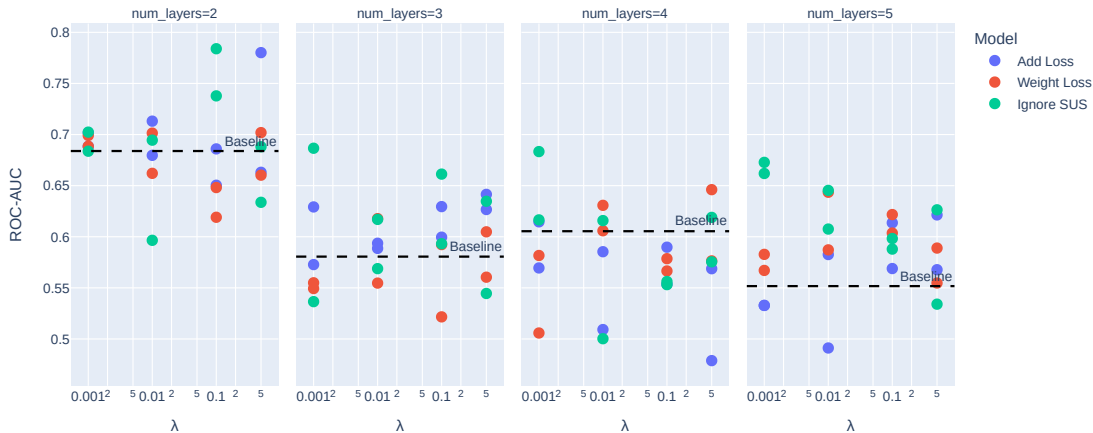
Variant 3 – Ignoring suspicious nodes:

$$L_{\text{total}} = \frac{1}{n} \sum_{i=1}^n (\|z_i - c\|^2 * (1 - L_{\text{cons}}(i)))$$

Here, $L_{\text{cons}}(i)$ denotes the soft rule violation for node i .

Results of first experiments

Effect of λ on ROC-AUC



- Implement the already developed “constraint score” that indicates the predicted class based on the constraints and how much a node would need to change to switch its class
- Experiment with integrating soft logic in different ways: in the loss function, as a node or graph feature, or as an additional term in the message passing
- Evaluate the use of hard logical rules
- Apply the approach to OCGTL

Thank you for your attention!

-  Badreddine, Samy et al. (Feb. 2022). “Logic Tensor Networks”. In: *Artificial Intelligence* 303, p. 103649. ISSN: 0004-3702. DOI: 10.1016/j.artint.2021.103649. URL: <http://dx.doi.org/10.1016/j.artint.2021.103649>.
-  Giunchiglia, Eleonora, Mihaela Catalina Stoian, and Thomas Lukasiewicz (July 2022). “Deep Learning with Logical Constraints”. In: *Proceedings of the Thirty-First International Joint Conference on Artificial Intelligence*. IJCAI-2022. International Joint Conferences on Artificial Intelligence Organization, pp. 5478–5485. DOI: 10.24963/ijcai.2022/767. URL: <http://dx.doi.org/10.24963/ijcai.2022/767>.
-  Stoian, Mihaela Cătălina et al. (2024). *How Realistic Is Your Synthetic Data? Constraining Deep Generative Models for Tabular Data*. arXiv: 2402.04823 [cs.LG]. URL: <https://arxiv.org/abs/2402.04823>.
-  Wang, Xuhong et al. (2020). “OCGNN: One-class Classification with Graph Neural Networks”. In: *CoRR* abs/2002.09594. arXiv: 2002.09594. URL: <https://arxiv.org/abs/2002.09594>.

-  Xu, Jingyi et al. (2018). *A Semantic Loss Function for Deep Learning with Symbolic Knowledge*. arXiv: 1711.11157 [cs.AI]. URL: <https://arxiv.org/abs/1711.11157>.
-  Yang, Zhun, Joohyung Lee, and Chiyoun Park (2023). *Injecting Logical Constraints into Neural Networks via Straight-Through Estimators*. arXiv: 2307.04347 [cs.AI]. URL: <https://arxiv.org/abs/2307.04347>.